

Lec 12 Types of Autoencoders: Architecture and Code-Level Understanding

Lec 12 Types of Autoencoders: Architecture and Code-Level Understanding

Lecture	12 / 63 + intro
Duration	39:39
Captions	English (auto-generated)
Words	6,364
Video	vo2z-cWDLGE
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome to the lecture series on fundamentals of generative AI and large language models. We are in the week two and the so far we have seen the uh working mechanism of autoencoders theoretically. In the previous session we have seen mathematically how exactly autoencoders works. We have seen the reconstruction laws both for mean square error as well as the binary cross entropy. That is if you are using an uh the real valued features then if the input is real valued fe valued features then we'll be using an mean square error. If the input is uh binary then we'll be using a binary cross entropy.

[00:39] In this particular uh session we will be uh seeing with the different types of autoencoders. More specifically we'll be focusing on CNN based autoencoder. Then we will see something called as a shallow autoencoder and a deep autoencoder. Now why is that? We are first starting with CNN is in the previous session in the last two sessions when we saw theoretically and mathematically autoencoders we had taken tabulated data that is where the encoder and decoder were implemented

as multi-layer perceptron. Now what if the input is an image then how given input image how the model is going to reconstruct it. So if an input image uh is um image if our input uh uh data is an image then it is an CNN based autoencoder. So how

[01:34] exactly CNN based autoencoder works. So we know that irrespective of whatever may be the data if it is an autoencoder we will have an encoder we'll have a latent space and we'll have a decoder. So so far encoder and decoder were of multi-layer perceptron because of the input data. But now since the input data is an image so the encoder has to be an CNN based. So go back and then recall from our week one session. So in the week one towards the end of uh the week the last two sessions were on CNN. So uh we had done the revision there because it will be useful here. So we know that C in CNN the different components are we had convolutional layer then we had activation function RLU then we had

[02:21] a pooling uh layer. So if you look into this convolutional layer is mainly responsible for extracting the features and pooling layer is responsible for reducing the dimension uh special uh dimension. So here also because it is an autoencoder and the uh input uh is an image. Now at an encoder side we are using an convolution layer and we are using a pooling layer. In the pooling layer again we have seen from our week one we have different types of pooling.

[02:57] It can be an um max pooling, min pooling, average pooling but in autoencoder more specifically we will be using an max pooling. So working mechanism all that uh you know you can once again recall from our unit to one [snorts] uh uh then you can get back to this particular uh session. So here we have an encoder. Encoder uses a convolutional layer and it also uses the pooling layer. In the pooling more specifically max pooling and what is that it is responsible for? Because ultimately you see encoder has to extract the most important features uh and reduce the uh dimension that's a latent space right the output of convolution will be a latent space or latent code. So how it is possible? It is possible only with convolution layer

[03:46] and the pooling layer. So that is the architecture of encoder uses convolutional layer and pooling mainly to reduce the spatial dimension while extracting the features. Next comes the latent space representation or latent code. What is this? This is the output of the last convolution layer in our architecture. So whatever is the output of last convolutional layer will be the latent space uh representation. So here uh the output of convolutional layer um here um in the encoder serves as the compressed representation that is the representation a smaller representation where the most important features are represent. Then this latent space is passed to an decoder. So so far decoder was a multi-layer perceptron. Now

[04:35] decoder cannot be multi-layer because we have an um image as an input. So the decoder has to use the transposed convolution or we call it as uh deconvolutional mainly. Why? Because ultimately the aim of a decoder is to reconstruct the data back.

So which means if our input is of dimension 28 to 281. So the uh latent space will be definitely of smaller dimension. Here you see the latent space is of 7 + 7. So uh uh when you pass it to a decoder decoder has to reconstruct back. So which means it has to give back to 28 + 28. So which means decoder has to do an upsampling upsampling of data so that it will match to the original input. So what we are discussing is CNN based and in CNN based autoencoder

[05:27] input is an image and at an encoder we are using convolution layer and then the pooling layer. At the decoder side we are using an transposed convolution uh to upsample it. So let's uh uh take an example uh here. Uh this is an architectural diagram of an CNN based autoencoder. If you see here there is an input which is of dimension 28x28x1. And what is one? One here is a channel which means we are considering an binary image. uh and 28 to 28 is the dimension which is height and then the width of the image. So that is the input that we have considered that is a pass to an convolutional layer. So convo 2D because we are implementing this in keras. So in a convert 2D so go back and then recall from our uh week one. So in the CNN what

[06:20] is that we'll be passing we are seeing how many such kernels we should take and the size of a kernel. So here um for a convolution operation to perform the number of kernels are 64 and the size of each kernel is 3 + 3 and the activation function used is a rail and remember this concept padding equal to same which means we are applying a padding. So uh when we apply padding whatever is the dimension you will be retaining the uh same dimension towards the end of a convolution operation. So padding is the same for this we are passing an input.

[06:55] What input 28 x 28 x 1. So the resultant is given to this particular variable. So what will be the uh output here? The output since padding is used 28 x 28. How many such filters we have used? 64 such filters we have used. So that is the output we have obtained. Now that is passed to an pooling layer. So what is pooling? A pooling will always reduce the dimension. So um here for a pooling we require an window size. So here the window size is 2 + 2 because the window size is 2 + 2 definitely the input will be reduced by half. So input is 28 x 28 that is reduced by half which is 14 x 14 and 64 will remain the same. So this is the output of the first convolution layer. When I say convolution here it is

[07:51] both convolution as well as pooling. So that is given to um this particular variable. Now that that entire is again passed to a second convolutional layer. So what is the uh input that you are passing? It is 14 x 14 x 64. So that is passed um to an second convolution layer which has 128 kernels. Each kernel is of size 3 + 3. So since the padding is the same whatever the input you are passing the same input. So 14 x 14 but um here number of kernels are 120 uh filters are 128. So that's what we have obtained here and that is passed to an pooling layer. Next is pooling. So in the pooling again we are using a window size 2 + 2. Therefore you are reducing it by

half. So $7 + 7$ into 128. So now that is

[08:45] again passed to an convolution because what is the latent space the output of the last convolutional layer. So uh again we are passing as an um this is passed as an input to this. So in this convolution we are using a 256 padding same. So the filter um the input is $7 + 7$ but instead of 128 you have 256 that is what is this right? So uh and this entire is a convolutional uh uh function convolution operation. So you have defined this under a function conv autoencoder. Um this is an uh uh input um you know encoder that we have uh you seen. So at uh since the input is an image encoder will have convolution and then the pooling layer that's what we have seen it. Now what is the latent space? Latent space is always an

[09:40] compressed representation of an input. Input is 28 to 28. By the time we came to an latent space the input now here is $7 + 7$. Is it not? We have reduced from $28 + 28$ to $7 + 7$. We have reduced it. Now our latent space dimension is $7 + 7$ uh uh and how many such 256. Now this is passed as an input to an decoder. So when you pass this to pass as an input to a decoder decoder has to do what? It has to upsample it because ultimately it has to get get the 28 cross 28 um you know it has to give back the uh dimension same as the input. Now you are at $7 + 7$. So now we have to do upsampling. So now let us understand what is upsampling with a simple example and then we'll come back to the same uh uh program. So what is upsampling? Now

[10:37] whatever the image that you are seeing here it is a feature map obtained from a last convolutional layer. So um now let us assume the size uh um here now you have reduced from $28 + 28$ to $7 + 7$. From this $7 + 7$ you should reconstruct back to $28 + 28$ which means now you have to increase this $7 + 7$ to uh $28 + 28$. So how will you do that? The uh let's take uh the different techniques that are used for upsampling. One is called nearest neighbor upsampling. I'm giving a simple example. Later we'll connect back to the um image. Now let us assume the X is um that is this uh image. Let us assume it is of two cross two dimension which is having the value 1 2 3 4. Just for an example to understand what is nearest neighbor upsampling.

[11:31] What is our ultimate aim from this? You have to increase the size. If you have to increase the size by how much you want to increase, that is what is this upscaling factor. So it's a uh scaling factor I mean a sampling factor not scaling I'm sorry it is an upsampling uh factor. So s value is two. So because it is two how you are going to apply on this because it is uh two every of the value should be expanded by $2 + 2$. So now [clears throat] you see this is expanded by $2 + 2$. This two is expanded by $2 + 2$. This three you are expanding by $2 + 2$ and $2 + 2$. Why $2 + 2$? Because of this sc up sampling factor s is two. So you have uh um you know increased by $2 + 2$.

[12:25] Now when you increase by $2 + 2$ you it is like position position position then two. This is what ideally right now when you have a values uh like this in this either you can write one or you can fill zero right either you can write to one or you can

fill zero. If you fill all of these values by one it is called the nearest neighbor uh upsampling. So what will be the output? Uh the output here input is 2×2 and up uh sampling factor is two. So multiply each of this you get 4×4 . So that's the output 4×4 where uh nearest neighbor meaning whatever is this value the same you are going to copy in each of this place.

[13:16] This is uh uh the same value is being copied. That is what is nearest neighbor upsampling. Second now uh in that you can even put zero right. So in that position we can even put zero. If you have zeros in all of these it is called bed of nails upsampling or it is zero insertion. So uh is it not we have increased the size but there is one drawback of just upsampling factor. What is the drawback? Is there any learning here? Is there anything that the model has learned? It is just take the values and then put all those values. So or put zero values. So in this particular method there are no learnable parameters.

[14:02] Although from 7×7 you are increasing to 14×14 just an example. So there is no uh uh you know uh if you take a matrix of 7×7 and if you have a scaling factor two then you will have an uh resultant 14×14 . So even though you have increased but you have just increased without any learnable parameters there is nothing to learn here. So the this is not a good approach for upsampling. Then you have a second uh method. What is the second method?

[14:36] That is called a transposed convolution operation. What is a transposed convolution? Take the input. The input here is $1 \ 2 \ 0 \ 1$. And then we'll be requiring a kernel. So what is that kernel value? The kernel value v is it is just one uh kernel value which we have taken it as 2×2 . What is a transpose convolution? You take the input multiply it with uh kernel. So how is that you do? So and what will be the output size by doing this because our ultimate aim is to upsample which means you have to increase the dimension. So what is the formula? It is input plus kernel minus one. So what is input?

[15:17] Input is two and what is the kernel size? It is kernel is also 2×2 . So $2 \times 2 - 1$ will be 3. This is for height and this is for width. So you are getting 3×3 . Now from input image 2×2 we are increasing uh uh to 3×3 . Now let us understand how what is the math how the calculation is going to happen. We know that ultimately the output is um 3×3 . So which means in all of these positions you will have this zero. Now you you have to multiply input should be multiplied with kernel. What is input?

[15:56] One. One is multiplied with all of these values. So since the position of one is here on the top left. So one into you will have the values. So you will have so these four values will be uh replaced it with uh you can see 1 into 4 is 4 then 1 into 3 is 3 1 into 2 is uh 2 and 1 into 1 is 1. So what is this one? 1 is input. What is $4 \ 3 \ 2 \ 1$? It is kernel. So this is the value that we have obtained which is G1. Similar way you take the second second input two that is getting multiplied with. So initially it will be all

zeros. So uh since two is uh in the top right. So top right four positions are all um you know zeros. So when two gets multiplied 4 2 into 8 6 then 2 into 2 4 2 with 1 is 2.

[17:07] So this is what is G2. Similar way you do it for uh the third input multiplied with the kernel and the fourth input multiplied with the kernel. So this is what uh you are going to multiply and towards the end the transpose convolution is you have got four different uh matrix combine all of those how is that you will do you will just perform an addition. So how is that you do you this is G1 this is G2 this is G3 and G4 and what is that you are doing it is $4 + 0 + 0 + 0$ is 4 then $3 + 8 + 0 + 0$ is 11 0 6 0 0 is 6 Respective positions you will do the addition it is 2. Similar way $1 + 4 + 0 + 4$ is 9. so on so forth. You will do an element wise addition in each of these matrix and you have obtained the transpose convolution.

[18:10] Okay. Now here uh there is something to learn right. What is there to learn? Kernel is there definitely initially the kernel values will be some random values over a period of time in the back propagation you're going to update these kernel values. Okay. Uh so which means from $2 + 2$ you have increased to $3 + 3$ which means you have upsampled it. So definitely better than the previous upsampling right where you have just copied the values. This you have performed some operation but there is one drawback here. What is that drawback? If you if you observe here um you know uh there is a kind of a problem that you can observe in this matrix. Uh what is that problem? Can you see here? There are some values which

[18:57] are very large. There are some values which are very very small. Right? So uh what is this problem called? It is called a checkerboard uh problem. What is this checkerboard problem? You can visualize this in the diagram. You can see there are some uh grid kind of a uh texture that you can able to see in the image where uh you have an uh you know um box or alternative bright or dark pixels or a square pattern. Can you able to see uneven texture? You you can able to see something like this as an output if you go for transposed convolution.

[19:36] And what is this problem? This is called a checkerboard problem. And this checkerboard problem has occurred because of uneven movement of the kernel onto this input uh uh image. So for some part of an input image you are applying kernel again. For some part of an input image you are applying kernel only once. So because of this you are able to see some square kind of a pattern. That is the problem with the transposed convolution. So which means although transposed convolution is increasing the dimension but we are facing this checkerboard problem. This is not intentional. It is not any noise which has occurred. It is the concept itself will create an um um you know uh there is a systematic artifacts which has

[20:26] occurred because of the operation itself not any uh noise or an external noise that has been added. No. How do you overcome this? So, uh first whatever is the input

perform an upsampling. How do you perform an upsampling? We can use either uh zeros or you can go for nearest neighbor zero insertion or nearest neighbor. So we have used nearest neighbor first perform upsampling then perform a convolution operation. So what is that? We have done first upsampling. How is that? We have done upsampling. This is 2×2 up scaling sampling factor is two. Therefore we have got 4×4 . So this is upsampling method. For this you are applying a convolution method. So what is convolution? Uh same as your previous

[21:14] one. You will take the kernel and you will multiply uh you will apply it onto an matrix and then perform a calculation. So what is that? First you will do you will take this 2×2 and you will take the entire kernel and then perform an uh operation. So uh how is uh that you are doing it you can see. So um it is one getting multiplied with this then one getting multiplied with this then this one is multiplied with this and this element wise you are performing a multiplication and then you calculate and then get the result uh and store it uh um as an output. Next uh um slide that you will take this part this is the next part. So uh this is what is and then you will take the same kernel perform element wise um

[22:07] multiplication and then this is one with this two with this then one with the two and two with one. So you are performing element wise and then you are getting a result and then you are storing it here. Similar way now when you scale you are going to slide when you you get this matrix. So you will take and then perform uh uh the same operation and so on you will be able to get the values. So once you complete when you uh stride you take this matrix then you take this then you take this again when you stride down you take this part then you take this part then you will take this part is it not systematically you have gone through the entire matrix here. So upsampling first spreads the values

[22:56] uniformly and the subsequent convolution when it is applied uh the is applied uh you are applying a kernel evenly. So this avoids an uneven overlap uh so that you here you will avoid that checkerboard artifacts. So uh if you see here um for the same uh uh input uh 1 2 0 1 and then the same kernel 4 3 2 1 uh the um matrix that you have obtained is this 4 16 29 50 21 and the same uh when you have performed upsampling and then when you have used the uh see the input is the same 1 2 1 here the input is the same and kernel 4 3 2 1 when you perform You are seeing that there is an even distribution not some values are very small some values are very large. No. So this is the concept that is used for um you know in

[23:54] autoencoders we first go for upsampling and then go for a convolution. You can see now what we are discussing is the decoder part. In the decoder part whatever the latent space is passed as an input. So the latent which is obtained from our previous one that is first you are doing an upsampling upsampling what is the uh up factor it is 2×2 . So now first you will increase it. So when you increase definitely if it is 7×7 it will be increased to 14×14 right. So because uh 7×7 when you take

a scaling factor as a two it is $14 + 14$. So that's what is a $14 + 14$. First you are upsampling it then you are performing a convolution operation. So convolution 128 such kernels are there.

[24:43] So uh um whatever is this uh 256 when you um perform an uh deconvolution operation you are going to get uh um uh from this uh you know the output of this will be $14 + 14$ uh with 128 because there are 128 such kernels right so this is what we have uh seen it so next uh the output of this is just see only so much 14 is changed to uh 17 7 is changed to $14 + 14$ but uh the number of kernels that will remain the same. But when this is passed as an input to this convolution operation the output what you get is $14 + 14$ uh with these many number of kernels 128 that's what is this uh uh value. Now again you are performing an upsampling. Upsampling again when you perform because the scaling factor is two $14 + 14$ will now

[25:45] be converted to $28 + 28$. So 14 is changed to 28. And next you are applying it to a convolution which is having 64. So it will be $28 + 28$ cross 64. This is the output that we have obtained. And you are using uh the last convolution layer. And why do we require a last convolutional layer? Because you see here the dimension is 28281. But here it is 282864. So now it should go back to 28281. So go for one um uh kernel. The kernel size is $3 + 3$ padding same. And then the output of the previous one will be passed as an input and then you are getting a reconstruction data back. So which we have obtained as 28 cross 281.

[26:37] Since it is an output, we have used sigmoid as an activation function. Why sigmoid? Because here the input we have in terms of zeros and ones. Uh because it's a binary, the data is in terms of zero and one. Definitely even at an output layer the data should be in terms of zeros and ones. Therefore the activation function used is sigmoid. So you will be seeing this in a hands-on session also but I'm trying to explain you how when an image is passed as an input to an convolution how autoencoder how autoencoder will um extract the features from features how it is going to reconstruct.

[27:17] So here uh the input what we have considered is this and then towards the end of autoencoder function we are going to get a latent output and then a reconstructed uh uh output. So we are going to get these and you can able to plot to visualize uh uh the results as well. So this is what is uh CNN based autoencoders where input is an image. Therefore encoder will have an CNN base that is convolution and pooling and the decoder will have upsampling and convolution. So this is what uh is and the sigmoid as I said we are using a sigmoid here because um the input features are in terms of binary ultimately at a reconstruction the output also should be in binary.

[28:02] Therefore we are using a sigmoid as an uh activation uh function. Now uh this is important concept to be uh remembered. Now the latent space is an hyper-parameter right. So how much dimension you want to reduce it that you will be in

the code itself you'll be mentioning it. If you have a smaller uh number, so what is going to happen if you have a smaller number? Is it not you're compressing information so much that um you know the information important information you have compromised because you're reducing too much and with that compromised uh uh latent space when you reconstruct back definitely the reconstructed image will not be very good. So it will be too much of blur uh uh when when you have a

[28:48] smaller latent space. So you will have a lot of blurriness but at the same time if you have if you increase the latent uh space but not as much as the size of the original input. No if the dimension of an latent space is increased um uh by a little larger number. Definitely the compressed information is very good with this. The reconstructed one is also very good. You'll get the reconstructed image quality also very good. But just because we get a very good reconstructed one by having a larger dimension, [snorts] if you increase it beyond a certain limit, you will start getting the diminishing results. You will start uh you know getting a lower quality um images. So therefore this is an hyperparameter and

[29:39] the latent space dimensionality will actually control what should be the reconstruction uh uh quality. [snorts] So the smaller poor larger it is good but just because larger is good if you go for higher and higher number then the quality of the image starts deteriorating. So from this you should know you cannot go for very smaller value nor you can go for larger you can fine-tune the parameter here. So this is all about the CNN based autoencoder.

[30:08] Let us see two simple other autoencoders. One is a shallow autoencoder. So what is a shallow autoencoder? The word itself says shallow. This is um where you have just one hidden uh layer. So when you just have one hidden layer, how does the architecture uh looks? You have an input layer. Input layer is passed to an hidden layer. The output of that hidden layer itself is the latent representation that is then passed to an output layer. So you just have only one hidden layer between input and output.

[30:41] So let us uh say one uh example here. So here the input is uh 784. So ultimately you want output also 784. Between input and output there is only one uh hidden layer and that hidden layer is um a hidden layer having a 32 neurons. So when 784 is passed to an hidden layer which is having a 32 the output will be 32. This 32 should be passed now to a layer because ultimately you should now pass to an output layer which should give you 784. So at an output layer you are passing 784 neurons. So let us understand that input is 784 that is passed uh to an uh hidden layer. Input is 784. Hidden layer is having 32 neurons and activation function is ru.

[31:34] So therefore encoded output will be having 32 that is passed as an input. Now uh um so 32 is passed as an input to this the how many neurons you should have 784. Why 784? Because it is an shallow where after hidden layer that is an output

layer. So at the end of an output layer you have to get 784. If you have to get 784 then you have to pass these many number of neurons. And what activation uh function we are using sigmoid. Why sigmoid we have to use?

[32:06] Because ultimately if our input is in terms of binary then the output also should be in terms of binary. Therefore the activation function what is used is sigmoid. Now uh uh what if our input is some numbers but see during our pre-processing what if you have scaled down the values between 0 and one you we have normalizing the data right when we scale down the value between 0 and 1 then ultimately the output also should be in terms of 01 then we'll be using a sigmoid. So at the output uh um at the decoder output the reconstructed value should match the range of an input data.

[32:47] Suppose if a pre-processing scales the features to 01 then obviously sigmoid is an activation function that you should use at an output layer and so that it will scale down and what should be the loss function? Definitely binary cross entropy should be the loss. Suppose you have a features where you have not scaled the values between zero and one. you are using a real valued only. If you are using a real valued or a standardization method then uh um you know use a linear activation here it should be a linear activation and the loss function will be a mean square error loss. So what is the shallow?

[33:26] Shallow autoencoder is the one where we just have only one hidden layer between input and output. Next we have deep autoencoder. What is deep autoencoder? The name itself suggest you just don't have one hidden layer, you have multiple hidden layers. So when you have multiple hidden layers, how does it works? Definitely you should go in the decreasing order at an encoder side and you should go in the increasing order at the decoder side. The output of last decoder will be the latent space. Latent space is passed as an input to the decoder. So you see input is 784 output is 784 which is first passed to an hidden layer which is having 128 that is then passed to a next hidden layer which is having 64 which is then passed to a

[34:12] next hidden layer which is having 32. Since it is the last uh layer the output of that will be the latent. Since it is 32, 32 will be the uh output that is then passed to an decoder which is having 64 then 128 then it is 784. Output of this will be a 784 values. So let us see how we have coded that 784 is passed as an input that is first passed to a first hidden layer H1 which is having 128. So 128 activation RLU. So that is given to X. Now that is passed as an input to the next which is having 64. So 64 that X is passed you are again storing the value back in X that is again given to a decoder which I mean sorry encoder which is having 32.

[35:01] So 32. Now this X is passed and whatever output you are getting will be an latent representation. Now this latent representation is passed as an input to a decoder which is having 64. So latent decoder having 64. Then that is passed to an next decoder hidden layer which is having 128 that is then passed to uh this is the

last layer which should have 784. Last layer 784 and sigmoid same if you have scaled down the values between zero and one then use a sigmoid.

[35:37] » [laughter] » uh or or if you have used a standardization where you have not scaled the values where it is a real valued then use a linear activation um function. So this is what is deep autoencoder. Now um can you think of this? Can I use a deep autoencoder uh architecture for an images? Uh we have seen some time back for an image we have to use for CNN based autoencoder. What uh what will happen if I use uh uh deep learning uh I mean a deep uh autoencoder. What is deep autoencoder?

[36:11] If you take this 784 this is $28 + 28 + 1$ which is the image which is having 28 28 you are converting that into a single vector of size 784. Now uh um so the input which is having uh 28 28 um you have converted into a single vector and then you have passed it to a deep autoencoder. Can we use this for an image? What is the answer? Definitely answer is no. Why? Because the minute you have converted that into this the special information is lost.

[36:45] When special information is lost look at this uh uh this is an output of deep autoencoder. This is the output of CNN autoencoder. Can you look at wherever I have highlighted it with the red? Can you look at this part and this uh uh so here can you see the features are more sharp here whereas um you have lost the information there and uh similar way can you able to see this image and this image uh can you able to see there is something that is there on the t-shirt but that is not being captured here properly. Similar way you can see this example and this example. Why all this is happening is because here you have flattened the image $28 + 28$ you have reduced I mean when you have flattened it you are getting a 784.

[37:34] So because of this it is failing to preserve the spatial relationship. The edges contours shapes are being lost when you use a deep uh um you know autoencoder. Whereas if you use CNN definitely because you are convolution the local spatial structures are preserved and then it learns the edges textures and different shapes and decoder uses are using the learned feature maps you can able to reconstruct the image uh properly. Therefore whenever we are dealing with an image as an input it is always advisable to go for CNN based autoencoder. If you have a tabulated data you can go for either a shallow or you can go for an deep autoencoders.

[38:22] With this we have come to an end of this particular uh uh session. In this session we have uh seen uh the different types of autoencoders and in this we have started with CNN based autoencoder. Then we have uh seen and shallow and then deep autoencoders. uh and in the upcoming session we will see something called as an regularization techniques in autoencoder. So there can be an high chances of overfitting a problem here. If there is an overfitting problem what are different regularization techniques that we will be using in autoencoder. So you can recall the concepts the various regularization techniques what we have already seen in our

deep learning course. We have seen L1 L2 regularization. We have seen dropouts.

[39:12] We have seen early stopping. Then we have something called as in batch normalization if you are dealing with an images. So all these are already the regularization techniques. So uh can we use those regularization techniques or do we have any other special regularization techniques for autoencoder? So that is that point of discussion for our next session. Thank you.